
Knowledge Distillation of GPT-2 Small

Yicheng Yao

Electrical & Computer Engineering
University of Toronto

yicheng.yao@mail.utoronto.ca

Jarvis Wang

Electrical & Computer Engineering
University of Toronto

jarvis.wang@mail.utoronto.ca

Jiangchuan Yu

Electrical & Computer Engineering
University of Toronto

jiangchuan.yu@mail.utoronto.ca

Abstract

Large language models are powerful but expensive to run, so we wanted to see how much of GPT-2's ability survives when it is shrunk to a quarter of its size. We use a pretrained GPT-2 small as the "teacher" (124.4M parameters) and train a much smaller "student" (30.3M parameters) from scratch on WikiText-2. The architecture, data, and training loop are held fixed, and we compare three ways of training the student that differ only in the loss: learning from the real text (corpus only), from the teacher's soft predictions (teacher only), or from both. We also sweep the distillation temperature. Using the teacher helps a lot, roughly halving the test perplexity compared with training on the text alone. The teacher-only student came out best, at a test perplexity of 90.7, ahead of the combined loss (115.6) and well ahead of corpus-only (227.2). The student ends up about $4.1\times$ smaller than the teacher, a quarter of the disk size, and roughly $1.8\times$ faster to generate, at the cost of higher perplexity.

Attestation of Teamwork

All three members contributed to the project. **Yicheng Yao** wrote the distillation loss functions (the three objectives, the temperature scaling, and the T^2 correction), the training loop with early stopping and best-weight restoration, and the temperature sweep. **Jarvis Wang** built the data pipeline (tokenizing and blocking WikiText-2), set up the teacher and student models, and wrote the evaluation code for perplexity, text generation, and speed. **Jiangchuan Yu** ran all the experiments, made the figures and tables, and led the writing of this report and the slides.

1 Introduction

Modern language models work well but are heavy to deploy. They are slow, use a lot of memory, and cost money to serve, which is a real problem when a model has to run on a phone, on cheap hardware, or under a tight latency budget. **Knowledge distillation** [1] is a well-known way to make a model smaller without starting over: we train a small *student* model to copy a larger *teacher* model.

What makes distillation work is that the teacher gives more than just the right answer. For every next word it produces a full probability distribution over the vocabulary, and even when the top choice is obvious the smaller probabilities it assigns to the other words are informative. Hinton et al. call this extra signal *dark knowledge*, and a student that learns from it can pick up how the teacher generalizes rather than just which single word is correct.

We apply this idea to GPT-2 small on WikiText-2. The aim is to measure, in a controlled setup, how much of the teacher's language ability transfers into a much smaller student and what we get back in speed and size.

2 Preliminaries and Problem Formulation

Setup. Let $P_T(\cdot \mid x_{<t})$ and $P_S(\cdot \mid x_{<t})$ be the next-token distributions of the teacher and student given the words so far. The teacher is a frozen, pretrained GPT-2 small; the student is a smaller GPT-2-style model trained from scratch. They share the same GPT-2 tokenizer (50,257 tokens) and the same 1024-token context, because to compare or copy distributions the two models must be talking about the same tokens at the same positions.

Goal. Train smaller and faster student models that still perform well on unseen validation and test text. The following five questions help us compare model quality and efficiency:

1. Does using the teacher beat plain training on the text?
2. Is the teacher signal *alone* enough, or do we still need the real labels?
3. Which temperature transfers the teacher’s knowledge best?
4. How far behind the teacher does the student end up?
5. How much smaller and faster do we get?

Perplexity. We measure quality with perplexity, $\text{PPL} = \exp\left(\frac{1}{N} \sum_t -\log P(x_t \mid x_{<t})\right)$, which is just the exponential of the average per-token cross-entropy on the real next words. Lower is better.

3 Design

Models. The teacher is gpt2 (GPT-2 small): 12 layers, hidden size 768, 12 heads, 124.4M parameters. The student keeps the same style but halves both the depth and the width: 6 layers, hidden size 384, 6 heads, 30.3M parameters.

Where the parameters come from. It helps to split a GPT-2 model into two parts. The **embedding tables** hold one vector of length d (the hidden size) for every token and every position, so they cost about $(V + L_{\text{ctx}}) \times d$ parameters, where $V = 50,257$ and $L_{\text{ctx}} = 1024$. Following the parameter-count approximation from Kaplan et al. [3], each standard **Transformer block** has about $12d^2$ parameters, so a stack of n blocks has about $12nd^2$. This approximation covers the attention and feed-forward weights while leaving out small terms such as biases. Plugging in the numbers:

- Teacher: embeddings $(50,257 + 1024) \times 768 \approx 39.4\text{M}$; stack $12 \times 12 \times 768^2 \approx 85.1\text{M}$; total $\approx 124.4\text{M}$.
- Student: embeddings $(50,257 + 1024) \times 384 \approx 19.7\text{M}$; stack $12 \times 6 \times 384^2 \approx 10.6\text{M}$; total $\approx 30.3\text{M}$.

The key point is that the two parts shrink at different rates (Appendix A). The embeddings only halve, because their row counts (50,257 tokens, 1024 positions) are fixed by the shared tokenizer and context, so only the width d changes. The Transformer stack shrinks by $8\times$: halving the width quarters each block (d^2) and halving the depth removes half the blocks, giving $4 \times 2 = 8$. So the headline number is only $4.1\times$ even though the part that does the real work is $8\times$ smaller — which will matter for speed.

Three objectives, one pipeline. We train three students that are identical in every way (architecture, starting weights, data order) except for the loss. Writing CE for the cross-entropy against the true next word, the distillation term is

$$\mathcal{L}_{\text{KD}} = T^2 \cdot \text{KL}(\text{softmax}(z_T/T) \parallel \text{softmax}(z_S/T)),$$

where $\mathcal{L}_{\text{KD}}$ is the KL-divergence-based knowledge-distillation loss, z_T and z_S are the teacher and student logits, and T is the temperature. The three objectives are:

Corpus only: $\mathcal{L} = \text{CE}$; Teacher only: $\mathcal{L} = \mathcal{L}_{\text{KD}}$; Teacher + corpus: $\mathcal{L} = \alpha \text{CE} + (1 - \alpha) \mathcal{L}_{\text{KD}}$,

with $\alpha = 0.5$. The pipeline is the same every time: run the batch through the frozen teacher and the student, compute one loss, and update only the student.

The T^2 factor. Dividing the logits by T softens both distributions, but it also shrinks the soft-target gradients by about $1/T^2$. Following Hinton et al. [1], we multiply the KL term by T^2 so the teacher signal keeps a steady strength as we change the temperature, and stays on the same scale as the cross-entropy term in the combined loss.

4 Methodology

Data. We use `wikitext-2-raw-v1` from Hugging Face (about 2.4M GPT-2 tokens) for everything. We tokenize the text, join it into one long stream, and split it into non-overlapping 1024-token blocks. GPT-2 was not trained or fine-tuned on WikiText-2, so the teacher is tested zero-shot. The students are trained on WikiText-2, which gives them an advantage. A teacher trained on the same data would likely have lower perplexity, so the actual gap between the teacher and students may be larger.

Training. Each student is trained with AdamW ($\text{lr} = 5\text{e-}4$), batch size 16 (16,384 tokens per step), for up to 50 epochs with early stopping (patience 2). After each epoch, we check validation perplexity and save the model weights when it improves. For final testing, we use the weights from the epoch with the lowest validation perplexity, which is the best validation checkpoint. To make the three-way comparison fair, the training function re-seeds itself on entry (seed 42), so all three students see the same starting weights, the same batch order, leaving the loss as the only difference.

Temperature sweep. Temperature is important because it changes what the student learns from the teacher. A low temperature mainly focuses on the teacher’s most likely token, while a higher temperature shows more information about other possible tokens. However, if the temperature is too high, the distribution becomes too flat and the teacher signal becomes less useful. The best value depends on the teacher, student size, and dataset, so it cannot be chosen confidently without testing. This experiment shows how sensitive our distillation method is to temperature and helps us choose the strongest teacher signal for the final comparison. We test $T \in \{1, 2, 4, 7, 10, 15\}$. Each sweep run trains a throwaway student for 5 epochs with the **teacher-only** loss, so temperature is the only factor changing the gradient. We do not fully train every sweep model because the goal is only to rank the temperatures under the same short training budget. We choose the value with the lowest validation perplexity and use it for the longer final training runs. A limitation is that the ranking could change if every sweep model were trained for more epochs.

Implementation. PyTorch with Hugging Face transformers (GPT2LMHeadModel, GPT2TokenizerFast) and datasets. The KL term uses `F.kl_div` with `reduction = "batchmean"`, which averages over tokens so it sits on the same scale as the mean cross-entropy. GPT-2 uses the output at each position to predict the following token. Therefore, we shift the logits and labels by one position: the prediction at position t is compared with the token at position $t + 1$. Without this shift, the model would be trained to predict the token it already received instead of the next token. Runs used an AMD Radeon PRO W7900 (48 GB, ROCm).

5 Numerical Experiments

The settings shared by every run are listed in Appendix B.

Picking the temperature. Figure 1 shows the sweep. Under the teacher-only loss, validation perplexity is lowest at $T = 1$ and climbs steeply as the temperature goes up, from about 258 at $T = 1$ to about 2191 at $T = 15$. The curve is basically flat and bad past $T = 10$. The reading is that softening the teacher hurts here rather than helps, so we set $T = 1$ for the final students. (The absolute numbers are high because each sweep run is short and never sees the real labels; we only use them to *rank* the temperatures, not as final scores.) We say more about *why* $T = 1$ wins in the Discussion.

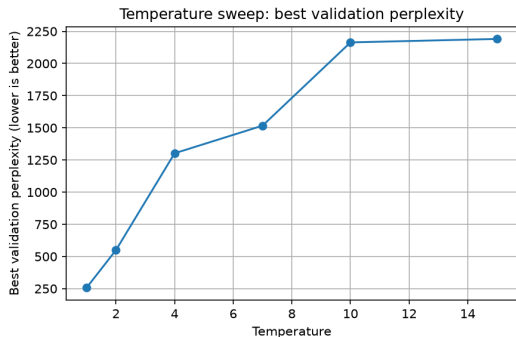


Figure 1: Temperature sweep result

The main comparison. Table 1 summarizes the final results, while Figure 2 shows the training curves, test perplexity, and generation speed. The three students use the same architecture, initial weights, data, and training settings. Their training objective is the main difference, so the results show how each loss affects the same student model.

The validation perplexity panel in the upper right of Figure 2 gives the clearest comparison during training. All three students improve quickly during the first few epochs, but their results later separate. Corpus only reaches its best validation perplexity of 214.5 at epoch 13 and then begins to get worse. Teacher + corpus continues improving until epoch 22 and reaches 111.2. Teacher only improves until epoch 39 and reaches 89.4, which is the best student result. The circles show the checkpoint selected for final testing. These curves suggest that teacher guidance helps the student train longer before overfitting. The training loss panel in the upper left should not be compared directly because Corpus only uses cross entropy, Teacher only uses KL divergence, and Teacher + corpus uses a weighted mixture of both losses.

The test perplexity panel in the lower left shows the final quality of each model. GPT 2 has the lowest test perplexity at 29.4. Among the students, Teacher only performs best at 90.7, followed by Teacher + corpus at 115.6 and Corpus only at 227.2. Compared with Corpus only, Teacher + corpus lowers test perplexity by about 49 percent, while Teacher only lowers it by about 60 percent. This shows that the teacher distribution gives the student useful information that is not available from the true next token alone.

The generation speed panel in the lower right shows that all three students run at about 292 to 295 tokens per second, compared with 164.6 tokens per second for GPT 2. Each student has 30.3M parameters and uses about 119.1 MB on disk, while the teacher has 124.4M parameters and uses about 474.7 MB. The students have almost the same speed and size because they share the same architecture. Their training objective changes model quality but does not change inference cost.

Model	Params (M)	Disk (MB)	Val PPL	Test PPL	Tok/s	Best ep.
Teacher (GPT-2)	124.4	474.7	30.6	29.4	164.6	—
Student: Teacher only	30.3	119.1	89.4	90.7	292.3	39
Student: Teacher + corpus	30.3	119.1	111.2	115.6	295.1	22
Student: Corpus only	30.3	119.1	214.5	227.2	294.3	13

Table 1: Final results

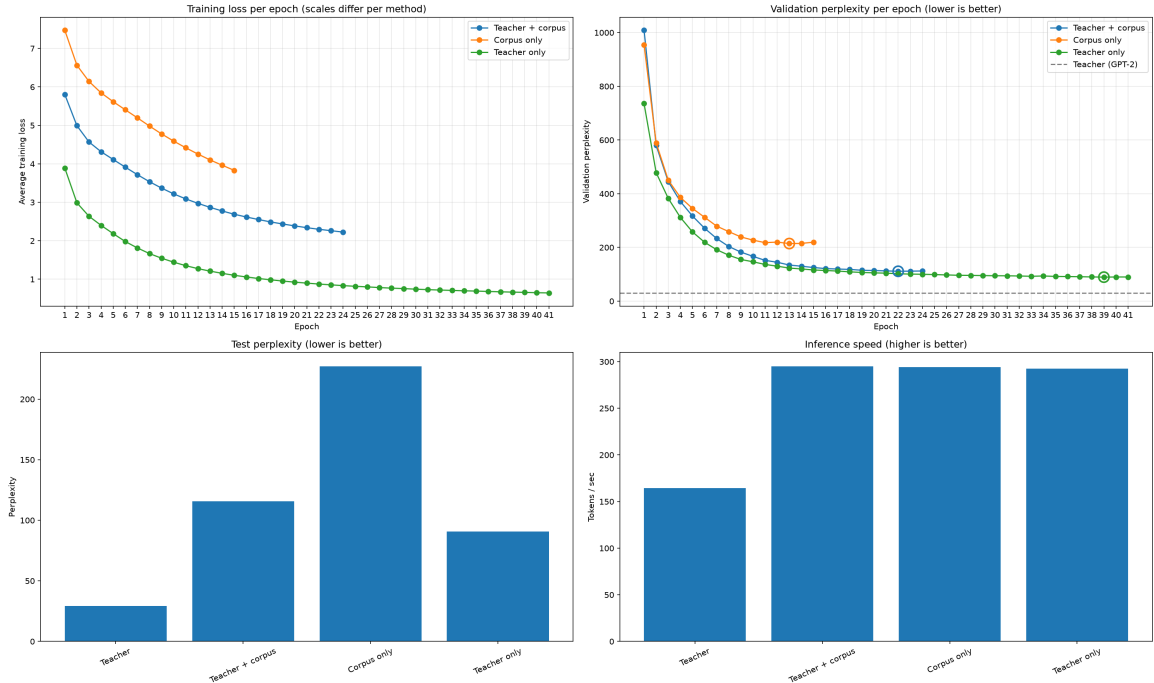


Figure 2: Full training results

What the text looks like. With greedy decoding, the teacher writes fluent sentences while all three 30M students fall into repetition, expected at this size. Still, the teacher-guided students are noticeably less broken than the corpus-only one, which collapses into tight loops (for example, repeating “city” over and over). A couple of transcripts are in Appendix C.

6 Discussion

Here we answer the five questions and address points that came up around the presentation.

Q1 — Does distillation help? Yes. Both teacher-guided students roughly halve the corpus-only test perplexity (90.7 and 115.6 vs. 227.2).

Q2 — Is the teacher alone enough? It was not just enough, it was the best, which we did not expect this at the beginning. We think the reason is overfitting. Our corpus is small, and the hard labels push the student to memorize exact next words, so the corpus-only run tops out at epoch 13 and the combined run at 22. The teacher’s soft distribution is a gentler, information-rich target that is harder to memorize, so the teacher-only student keeps improving until epoch 39. This lines up with Hinton et al.’s observation that soft targets act like a regularizer [1]. The combined loss sits in between because it still carries some of the hard-label pressure.

Q3 — Why does $T = 1$ win? The best temperature drops as the student gets smaller. A high temperature asks the student to match the teacher’s whole 50,257-way distribution, but a 30M model does not have the capacity for that; it is better off spending its capacity getting the top token right, which a sharp ($T = 1$) target emphasizes. Hinton et al. show exactly this trend for small students [1], and here the sweet spot lands at the edge of our grid, $T = 1$.

Q4 — How far behind the teacher? A fair way behind (90.7 vs. 29.4), and the gap is actually *understated*: the teacher is judged zero-shot (it was never trained on WikiText-2) while the students train in-domain, so a teacher fine-tuned on WikiText would look even better. This is unsurprising given the $8\times$ smaller Transformer stack.

Q5 — What do we gain? $4.1\times$ fewer parameters, about $4\times$ smaller on disk, and about $1.8\times$ faster generation. The speedup is smaller than the parameter ratio because GPT-2 ties the output layer to the token-embedding table, so every generated token still pays for a projection over all 50,257 tokens (the part that only halved), and at batch size 1 generation is limited by latency, not raw compute.

7 Conclusions

Our results show that knowledge distillation can greatly improve a smaller GPT-2 model. The Teacher only student achieved a test perplexity of 90.7, followed by Teacher + corpus at 115.6 and Corpus only at 227.2. This means that both methods using teacher guidance performed much better than training only on the true next tokens. Teacher only also gave the best student result in our setup. The temperature sweep selected $T = 1$, showing that a sharp teacher distribution worked better than softer distributions for this student. However, all students still performed worse than the GPT-2 teacher, which achieved a test perplexity of 29.4.

The student also provided a clear efficiency improvement. It used 30.3M parameters instead of 124.4M, required about 119 MB instead of 475 MB on disk, and generated about $1.8\times$ faster than the teacher. Teacher only continued improving until epoch 39, while Corpus only reached its best result at epoch 13. This suggests that the teacher distribution helped reduce overfitting on the small WikiText-2 dataset. Overall, the project shows that distillation can provide a useful balance between model quality, size, and generation speed. Our experiment was limited by one random seed, a small dataset, a fixed α , and a short temperature sweep.

Future work should first repeat the experiments with several random seeds and report the average and variation of the results. This would show whether the differences between the three methods are reliable. We could also tune α together with T instead of fixing $\alpha = 0.5$. This would test whether Teacher + corpus could perform better with a different balance between the two losses. Training on WikiText-103 would provide more data and may reduce overfitting. A teacher fine-tuned on the same dataset would also give a stronger and fairer upper bound. Testing several student sizes would show how quality, speed, and model size change as student capacity increases.

References

- [1] G. Hinton, O. Vinyals, and J. Dean. *Distilling the Knowledge in a Neural Network*. arXiv:1503.02531, 2015.
- [2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv:1910.01108, 2019.
- [3] J. Kaplan et al. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.

Appendix

A. Parameter breakdown. How the counts in the Design section split between the embedding tables and the Transformer stack.

Table 2: Parameter breakdown (millions).

Component	Teacher	Student	Ratio
Embedding tables	39.4	19.7	$2.0\times$
Transformer stack	85.1	10.6	$8.0\times$
Total	124.4	30.3	$4.1\times$

B. Experimental configuration. The settings shared by all training runs.

Table 3: Experimental configuration.

Setting	Value	Setting	Value
Dataset	wikitext-2-raw-v1	Optimizer	AdamW, lr $5e-4$
Block size	1024	Epochs / patience	50 / 2
Batch size	16	Sweep epochs	5
Teacher	GPT-2 small (124.4M)	α	0.5
Student	6L / 384 / 6h (30.3M)	Temperatures	1,2,4,7,10,15

C. Sample generations (greedy decoding). The teacher is fluent; the corpus-only student loops tightly; the teacher-guided students are rough but less degenerate.

Table 4: Representative greedy generations (truncated).

Prompt / Model	Continuation
“The city was famous for its”	
Teacher	... high-speed rail system, which was built in the 1930s. The city’s first subway was built in the 1930s. ...
Teacher only	... “greatest and most beautiful and beautiful, but also a great example of the most beautiful. ...”
Teacher + corpus	... founding in the city of the city of the city of the city of the city...
Corpus only	... city . The city ’s city ’s city is a city , and the city ’s city ...
“Once upon a time in a small village”	
Teacher	... a young girl named Nana was kidnapped by a group of bandits. She was taken to a small village. ...
Teacher only	... and the village’s the village’s had been evacuated to the village.

Prompt / Model	Continuation
Teacher + corpus	... of the village of the village of the village... John 's village.
Corpus only	... The first phase of the area of the area of the area of the area. . .